1. Problem Statement

Small organisations require a dependable network through which wired employees, wireless users, internal servers, and authorised external users can communicate while maintaining separation between different device groups. A single flat broadcast domain makes it harder to organise traffic, apply service-specific access restrictions, and maintain connectivity when a network link fails. This project addresses that problem by designing and documenting a small-company network in Cisco Packet Tracer. The design separates wired clients, servers, wireless clients, and network management into VLANs; connects the VLANs through IEEE 802.1Q trunking and router-on-a-stick routing; and uses DHCP, DNS, HTTP/HTTPS, ACL filtering, NAT/PAT, and Rapid-PVST to provide network services, controlled access, external web publishing, and redundant Layer 2 paths. A separately described Python HTTP/1.1 server demonstrates application-layer request handling over TCP.

2. Introduction

Computer networks allow connected devices to exchange data and share services through physical links, switching, addressing, routing, and application protocols. The project models a small enterprise environment containing wired desktop clients, wireless users, web and DNS servers, Layer 2 switches, an enterprise gateway router, an ISP router, and an external test computer. Its purpose is to show how these components cooperate to provide access to an internal website while keeping network functions organised and applying basic security and availability mechanisms.

The internal network is divided into four Virtual Local Area Networks (VLANs): VLAN 10 for wired clients, VLAN 20 for servers, VLAN 30 for Wi-Fi clients, and VLAN 99 for management. A VLAN creates a separate Layer 2 broadcast domain on shared switching infrastructure. IEEE 802.1Q trunk links carry traffic for multiple VLANs between the core and access switches and between the core switch and R1. VLAN 99 is configured as the native VLAN on these trunks; native-VLAN frames are untagged, whereas frames belonging to the other allowed VLANs are carried with 802.1Q tags.

Inter-VLAN communication is provided by Router-on-a-Stick. R1 uses subinterfaces on FastEthernet 0/0, each configured with an 802.1Q VLAN identifier and the gateway address for that VLAN's IPv4 subnet. R1 also provides Dynamic Host Configuration Protocol (DHCP) service to the wired and wireless client networks. DHCP supplies clients with network configuration, including an IP address, default gateway, and DNS-server address. The DNS server resolves www.project.local to 192.168.20.10, allowing users to reach the web server by name rather than entering its numeric IP address.

The web server provides HTTP and HTTPS on TCP ports 80 and 443. An extended ACL named CLIENT-ACL is applied inbound on the client and Wi-Fi router subinterfaces. It permits DHCP server traffic, UDP DNS queries to the DNS server, HTTP and HTTPS to the web server, and ICMP test traffic; it then denies other traffic destined for the server subnet and permits traffic to other destinations. NAT overload (PAT) translates internal addresses to R1's outside interface address for outbound connectivity. Static TCP NAT entries forward external HTTP and HTTPS requests to the internal web server. Wireless devices join the ProjectNet SSID using WPA2-PSK authentication with AES encryption.

The switching topology contains a core switch and two access switches connected by redundant Layer 2 paths. Rapid Per-VLAN Spanning Tree (Rapid-PVST) prevents switching loops and permits an alternate path to become active after a link failure; SW-CORE is assigned a lower bridge priority to become the root bridge for the configured VLANs. Additional safeguards include sticky-MAC port security on the web server port, BPDU Guard on wired client ports, and SSH version 2 for remote device management. The project description also specifies a separate Python HTTP/1.1 server that accepts TCP connections, parses requests, requires the Host header, supports GET, HEAD, and POST, uses persistent connections with an idle timeout, returns specified HTTP status codes, and handles clients using threads. The Python source file was not included in the submitted materials; therefore, its behaviour is described from the project specification rather than independently verified from source code.

3. Network Diagram

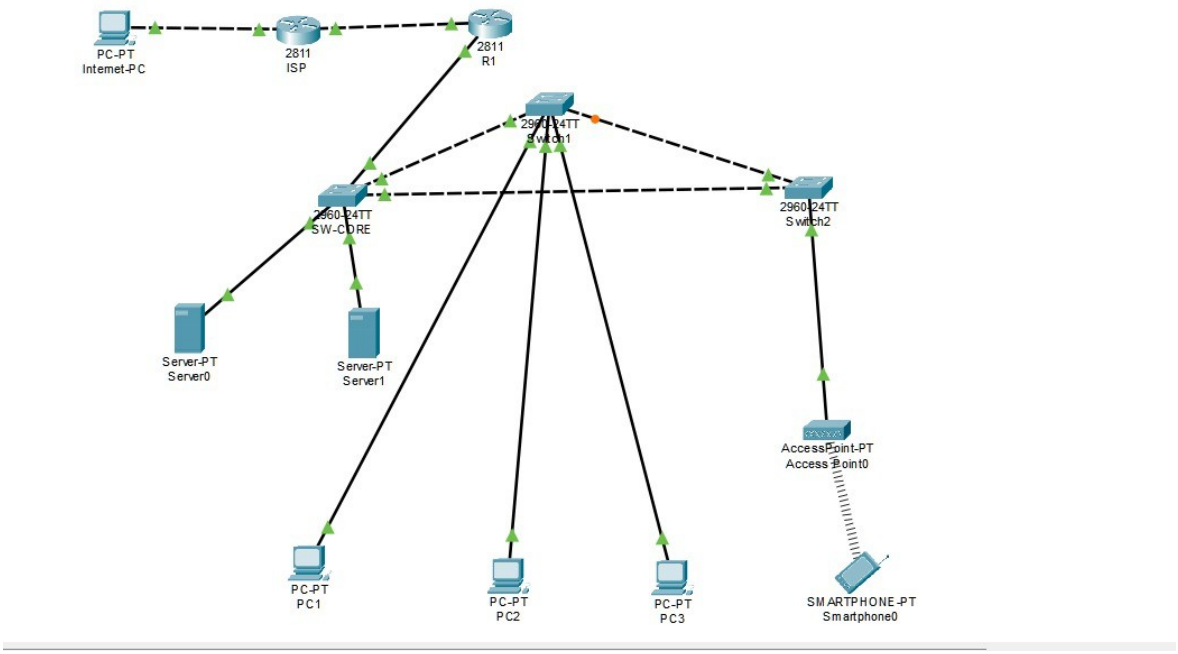


Figure 1. Physical Packet Tracer topology supplied for the project.

The topology places the Internet-PC on the external 198.51.100.0/24 segment connected to the ISP router. The ISP router connects to R1, which serves as the enterprise gateway. R1 connects to SW-CORE through the internal trunk. SW-CORE connects directly to Server0 (web) and Server1 (DNS), and has trunk links to SW-ACC1 and SW-ACC2. SW-ACC1 connects PC1, PC2, and PC3 in the wired client segment. SW-ACC2 connects the access point, which provides wireless access to Smartphone0. A further trunk between the two access switches supplies the redundant Layer 2 path. The diagram depicts physical cabling; VLAN membership, IP addressing, and routing are defined by the device configurations.

4. Configuration Setup

This section records the addressing plan and the configuration commands supplied for each network device, followed by the end-device settings. The interface names and addresses correspond to the submitted fix guide. Commands are presented in configuration order; device-specific secrets are retained as supplied because they form part of the lab configuration, but should be changed before deployment outside a controlled academic simulation.

4.1 Addressing Plan

Network / device	Address	Mask / gateway / purpose
VLAN 10 – wired clients	192.168.10.0/24	Gateway 192.168.10.1; DHCP clients
VLAN 20 – servers	192.168.20.0/24	Gateway 192.168.20.1; static server addresses
VLAN 30 – Wi-Fi	192.168.30.0/24	Gateway 192.168.30.1; DHCP clients
VLAN 99 – management	192.168.99.0/24	Gateway 192.168.99.1
R1 Fa0/1 – ISP link	203.0.113.2/30	ISP Fa0/1 = 203.0.113.1
Internet segment	198.51.100.0/24	ISP Fa0/0 = 198.51.100.1; Internet-PC = .10
Server0 – web	192.168.20.10	Gateway 192.168.20.1; DNS 192.168.20.11
Server1 – DNS	192.168.20.11	Gateway 192.168.20.1
SW-CORE management	192.168.99.2	Default gateway 192.168.99.1
SW-ACC1 management	192.168.99.3	Default gateway 192.168.99.1
SW-ACC2 management	192.168.99.4	Default gateway 192.168.99.1

4.2 SW-CORE – Core Switch

SW-CORE connects R1, both access switches, and the two servers. It carries VLANs 10, 20, 30, and 99 on its trunk ports and is configured as the spanning-tree root.

```
enable
configure terminal
hostname SW-CORE
no ip domain-lookup
spanning-tree mode rapid-pvst
vlan 10
 name Clients
vlan 20
 name Servers
vlan 30
 name WiFi
vlan 99
 name Management
exit
spanning-tree vlan 1,10,20,30,99 priority 4096
interface range fastEthernet 0/1 - 3
 switchport mode trunk
 switchport trunk native vlan 99
 switchport trunk allowed vlan 10,20,30,99
 no shutdown
exit
interface fastEthernet 0/4
 description Web-Server
 switchport mode access
 switchport access vlan 20
 switchport port-security
 switchport port-security maximum 1
 switchport port-security mac-address sticky
 switchport port-security violation shutdown
 spanning-tree portfast
exit
interface fastEthernet 0/5
 description DNS-Server
 switchport mode access
 switchport access vlan 20
 spanning-tree portfast
exit
interface vlan 99
 ip address 192.168.99.2 255.255.255.0
 no shutdown
exit
ip default-gateway 192.168.99.1
ip domain-name project.local
username admin secret Admin@123
crypto key generate rsa modulus 1024
ip ssh version 2
line vty 0 15
 login local
 transport input ssh
exit
service password-encryption
enable secret Class@123
end
write memory
```

4.3 SW-ACC1 – Wired Access Switch

SW-ACC1 connects PC1, PC2, and PC3 through access ports in VLAN 10. Its uplinks to SW-CORE and SW-ACC2 are trunks. BPDU Guard is enabled on the wired client ports.

```
enable
configure terminal
hostname SW-ACC1
no ip domain-lookup
spanning-tree mode rapid-pvst
vlan 10
 name Clients
vlan 20
 name Servers
vlan 30
 name WiFi
vlan 99
 name Management
exit
interface range fastEthernet 0/2 - 3
 switchport mode trunk
 switchport trunk native vlan 99
 switchport trunk allowed vlan 10,20,30,99
```

```
 no shutdown
exit
interface range fastEthernet 0/4 - 6
 description PC-ports
 switchport mode access
 switchport access vlan 10
 spanning-tree portfast
 spanning-tree bpduguard enable
exit
interface vlan 99
 ip address 192.168.99.3 255.255.255.0
 no shutdown
exit
ip default-gateway 192.168.99.1
ip domain-name project.local
username admin secret Admin@123
crypto key generate rsa modulus 1024
ip ssh version 2
line vty 0 15
 login local
 transport input ssh
exit
service password-encryption
enable secret Class@123
end
write memory
```

4.4 SW-ACC2 – Wireless Access Switch

SW-ACC2 connects the wireless access point through an access port in VLAN 30. Its two uplinks are trunks carrying the four configured VLANs.

```
enable
configure terminal
hostname SW-ACC2
no ip domain-lookup
spanning-tree mode rapid-pvst
vlan 10
 name Clients
vlan 20
 name Servers
vlan 30
 name WiFi
vlan 99
 name Management
exit
interface range fastEthernet 0/2 - 3
 switchport mode trunk
 switchport trunk native vlan 99
 switchport trunk allowed vlan 10,20,30,99
 no shutdown
exit
interface fastEthernet 0/1
 description Access-Point
 switchport mode access
 switchport access vlan 30
 spanning-tree portfast
exit
interface vlan 99
 ip address 192.168.99.4 255.255.255.0
 no shutdown
exit
ip default-gateway 192.168.99.1
ip domain-name project.local
username admin secret Admin@123
crypto key generate rsa modulus 1024
ip ssh version 2
line vty 0 15
 login local
 transport input ssh
exit
service password-encryption
enable secret Class@123
end
write memory
```

4.5 R1 – Enterprise Gateway Router

R1 performs inter-VLAN routing, DHCP service, client-side ACL filtering, NAT/PAT, and static web-port forwarding. Fa0/0 carries the internal VLAN trunk through subinterfaces; Fa0/1 connects to the ISP.

```
enable
configure terminal
hostname R1
no ip domain-lookup
interface fastEthernet 0/0
 no ip address
 no shutdown
exit
interface fastEthernet 0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 ip access-group CLIENT-ACL in
exit
interface fastEthernet 0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
exit
interface fastEthernet 0/0.30
 encapsulation dot1Q 30
 ip address 192.168.30.1 255.255.255.0
 ip nat inside
 ip access-group CLIENT-ACL in
exit
interface fastEthernet 0/0.99
 encapsulation dot1Q 99 native
 ip address 192.168.99.1 255.255.255.0
 ip nat inside
exit
interface fastEthernet 0/1
 description To-ISP
 ip address 203.0.113.2 255.255.255.252
 ip nat outside
 no shutdown
exit
ip dhcp excluded-address 192.168.10.1 192.168.10.9
ip dhcp excluded-address 192.168.30.1 192.168.30.9
ip dhcp pool CLIENTS
 network 192.168.10.0 255.255.255.0
 default-router 192.168.10.1
 dns-server 192.168.20.11
exit
ip dhcp pool WIFI
 network 192.168.30.0 255.255.255.0
 default-router 192.168.30.1
 dns-server 192.168.20.11
exit
ip access-list extended CLIENT-ACL
 permit udp any any eq bootps
 permit udp any host 192.168.20.11 eq domain
 permit tcp any host 192.168.20.10 eq 80
 permit tcp any host 192.168.20.10 eq 443
 permit icmp any any
 deny ip any 192.168.20.0 0.0.0.255
 permit ip any any
exit
access-list 1 permit 192.168.0.0 0.0.255.255
ip nat inside source list 1 interface fastEthernet 0/1 overload
ip nat inside source static tcp 192.168.20.10 80 203.0.113.2 80
ip nat inside source static tcp 192.168.20.10 443 203.0.113.2 443
ip route 0.0.0.0 0.0.0.0 203.0.113.1
ip domain-name project.local
username admin secret Admin@123
crypto key generate rsa modulus 1024
ip ssh version 2
line vty 0 4
 login local
 transport input ssh
exit
service password-encryption
enable secret Class@123
end
write memory
```

4.6 ISP Router

The ISP router represents the external network. Fa0/0 connects to the Internet-PC segment and Fa0/1 connects to R1.

```
enable
configure terminal
hostname ISP
no ip domain-lookup
interface fastEthernet 0/0
 description To-Internet-PC
 ip address 198.51.100.1 255.255.255.0
 no shutdown
exit
interface fastEthernet 0/1
 description To-R1
 ip address 203.0.113.1 255.255.255.252
 no shutdown
exit
end
write memory
```

4.7 Server Configuration

Device	IPv4 configuration	Enabled service / record
Server0 – Web	192.168.20.10; mask 255.255.255.0; gateway 192.168.20.1; DNS 192.168.20.11	HTTP ON; HTTPS ON
Server1 – DNS	192.168.20.11; mask 255.255.255.0; gateway 192.168.20.1; DNS 192.168.20.11	DNS ON; A record www.project.local → 192.168.20.10

The DNS A record maps the fully qualified host name www.project.local to the web server address 192.168.20.10. Both web protocols are enabled on Server0.

4.8 Client and Wireless Device Configuration

Device	Configuration
PC1, PC2, PC3	Select DHCP in Desktop > IP Configuration. Expected client subnet 192.168.10.0/24; gateway 192.168.10.1; DNS 192.168.20.11.
Access Point0	SSID ProjectNet; WPA2-PSK; passphrase Project@123; AES encryption. Ethernet connection to SW-ACC2 Fa0/1.
Smartphone0 / Laptop	Join ProjectNet using WPA2-PSK and the configured passphrase; use DHCP. Expected Wi-Fi subnet 192.168.30.0/24. A compatible wireless module is required for a laptop model without wireless support.
Internet-PC	Static IP 198.51.100.10; mask 255.255.255.0; default gateway 198.51.100.1.

The supplied physical topology screenshot visibly includes Smartphone0 as the wireless client. The project description also identifies a laptop as a supported Wi-Fi client; the screenshot does not show that laptop.

4.9 Python HTTP/1.1 Server Component

The project specification describes a separate Python program named server.py. It receives raw HTTP requests over TCP sockets, parses request lines and headers, requires the Host header, supports GET, HEAD, and POST, maintains persistent connections subject to an idle timeout, and uses one thread per client. The specified response codes are 200, 400, 403, 404, and 501. This component is distinct from the Packet Tracer server device: Packet Tracer demonstrates the network path and built-in HTTP/HTTPS service, whereas the Python program is intended to demonstrate HTTP/1.1 request processing. The server.py source file and runtime output were not included with the submitted files, so code-level verification and Python execution results cannot be claimed in this report.

5. Results

The submitted materials include the Packet Tracer topology image, the device-configuration and verification guide, and the Packet Tracer project file. The guide defines the tests and their expected outcomes. The Packet Tracer project file is a binary simulation artifact and could not be executed in the document-generation environment; no Packet Tracer command output, browser captures, ping captures, or post-failure screenshots were supplied. Accordingly, this section records the configuration-derived verification outcomes and the precise evidence criteria without representing unobserved tests as completed experimental measurements.

5.1 Configuration-derived verification results

Test / objective	Command or action	Outcome implied by the supplied configuration
Router interfaces	R1: show ip interface brief	R1 is configured with subinterfaces Fa0/0.10, .20, .30, .99 and outside Fa0/1. The expected state is up/up when the physical links and switch trunk are operational.
Trunking	SW-CORE: show interfaces trunk	SW-CORE Fa0/1–Fa0/3 are configured as trunks, native VLAN 99, allowed VLANs 10, 20, 30, 99.
Spanning tree	SW-CORE: show spanning-tree summary	Rapid-PVST is configured; priority 4096 is set for VLANs 1, 10, 20, 30, and 99. Root status must be confirmed from live output.
DHCP	PC1: ipconfig; R1: show ip dhcp binding	The CLIENTS pool serves 192.168.10.0/24; the expected client address is 192.168.10.10 or higher, with gateway .1 and DNS .11.
Inter-VLAN connectivity	PC1: ping 192.168.10.1; ping 192.168.20.10	The routing configuration provides the VLAN 10 and VLAN 20 gateway interfaces. Successful reachability depends on all links and server settings being operational.
DNS and web	PC1: ping www.project.local; browse to http://www.project.local and https://www.project.local	The DNS A record is configured to resolve the name to 192.168.20.10; Server0 has HTTP and HTTPS enabled. Successful application access must be confirmed in Packet Tracer.
ACL filtering	PC1: telnet 192.168.20.10	The ACL does not permit TCP/23 to the server subnet; the later deny statement therefore blocks this attempt.
External web access	Internet-PC browser: http://203.0.113.2	Static TCP NAT maps public TCP/80 to 192.168.20.10:80. The expected result is access to the web server if the ISP/R1 links and services are operational.
Redundant path	Disconnect SW-CORE Fa0/2; wait for convergence; retest connectivity	The inter-switch triangle provides an alternate Layer 2 path through SW-ACC2. Rapid-PVST is intended to reconverge; the actual transition and recovery time require live simulation evidence.

5.2 ACL behaviour and security scope

The ACL is applied inbound on R1 Fa0/0.10 and Fa0/0.30. Its sequence permits UDP traffic with destination port 67 (bootps), UDP DNS traffic to 192.168.20.11:53, TCP HTTP to 192.168.20.10:80, TCP HTTPS to 192.168.20.10:443, and ICMP. It then denies all remaining IP traffic destined for 192.168.20.0/24 and permits other IP traffic. Thus, the supplied configuration filters access to the server subnet, but it does not constitute complete isolation between client VLANs and VLAN 99. DNS over TCP/53 is not explicitly permitted. The ICMP permit is intentionally broad and is documented in the guide as a testing allowance.

5.3 NAT behaviour

The NAT overload rule matches source addresses in 192.168.0.0/16 and translates them using the address of R1 FastEthernet 0/1. The static TCP mappings forward public address 203.0.113.2 ports 80 and 443 to 192.168.20.10 on the corresponding ports. These are separate functions: PAT supports outbound address sharing, while static port translation publishes the internal web service to the external test network.

5.4 Evidence status

The supplied topology screenshot is included as Figure 1. The required minimum of three communication proof screenshots is not present in the submitted material. No screenshots have been fabricated or substituted for actual Packet Tracer output. The verification table above is therefore a technically grounded account of the configured behaviour and expected tests, not a claim that each test was observed to pass. This distinction is necessary for an academically accurate report.

6. Conclusion

The project designs a small enterprise network that brings together wired and wireless clients, internal servers, external connectivity, network segmentation, service filtering, and redundant switching. The configuration separates client, server, Wi-Fi, and management traffic into VLANs and carries them over 802.1Q trunks; Router-on-a-Stick routing on R1 provides inter-VLAN communication, while DHCP and DNS support automatic client configuration and name-based access to the web server. The extended ACL restricts client access to the server subnet to the specified services, NAT/PAT provides outbound address translation, and static TCP mappings publish the web server through the ISP-facing interface. Rapid-PVST is configured to manage redundant Layer 2 paths, while port security, BPDU Guard, and SSH provide additional infrastructure safeguards. The separate Python HTTP/1.1 component extends the project from network connectivity to application-layer request processing. The design demonstrates how multiple networking technologies combine to provide a structured and service-oriented network for a small organisation. Future development could add more granular management-plane controls, TCP DNS handling, stronger authentication, certificate-managed HTTPS, centralised monitoring, and measured performance and failover testing.